HARD Mock Set 1 — Accenture Technical (Above-Level)

Level: Harder than the 10 uploaded sets — every question needs 2+ reasoning steps, and most distractors are built around a real exam trap.

Syllabus: identical to Accenture (Common Apps & MS Office, Networking/Security/Cloud, DSA/DBMS/OOP/HTML-CSS-JS, Pseudocode, Applied SQL/Frontend) — only the depth is raised.

Format: 50 questions · Time: 50 minutes, closed book · No negative marking — attempt everything.

Section	Area	Qs
A	MS Office & Common Applications	8
B	Networking, Security & Cloud	12
C	CS Core — DSA / DBMS / OOP / HTML-CSS-JS	12
D	Pseudocode & Code Tracing	14
E	Applied — SQL & Code Output	4

Trace every pseudocode question **on paper with a variable table**. Do not do them in your head.

Section A — MS Office & Common Applications (8)

Q1. In Excel, `=VLOOKUP(102, A2:C10, 3, TRUE)` is used where column A is **not sorted** and contains 100, 105, 110. What is the result?

A) `#N/A`, because TRUE always requires an exact hit

B) The value from the row containing 105

C) The value from the largest entry ≤ 102 found by binary search — i.e. unreliable unless column A is sorted ascending

D) The value from the first row of the range

Q2. Cell **C4** contains the formula `=$A4+B$2`. It is copied to **D5**. What does D5 contain?

A) `=$A5+C$2`

B) `=$A4+C$2`

C) `=$B5+C$2`

D) `=$A5+C$3`

Q3. A1:A6 = 5, 12, 8, 20, 15, 4. What does `=COUNTIFS(A1:A6, ">=8", A1:A6, "<=15")` return?

A) 2

B) 4

C) 6

D) 3

Q4. `=IFERROR(VLOOKUP(7, A1:B5, 2, FALSE), "Not Found")` where 7 does not appear in A1:A5. Output?

A) 0

B) An empty string

C) `Not Found`

D) `#N/A`

Q5. A logo placed on the PowerPoint **Slide Master** appears on all 20 slides, but you must hide it on the title slide only. Best action?

A) Delete the logo from the master and paste it on 19 slides manually

B) Apply a different theme to the title slide

C) Insert the logo on the Notes Master instead

D) Select the title slide → Format Background → **Hide Background Graphics**

Q6. Two colleagues each edited their own copy of the same Word document. You want **one document containing both sets of edits** as tracked changes. Which feature?

A) Track Changes

B) Review → **Combine**

C) Review → Compare

D) Mail Merge

Q7. Which statement about `.csv` vs `.xlsx` is correct?

A) `.csv` can store multiple worksheets and cell formatting

B) `.csv` is a plain-text, single-table format with no formulas, formatting or multiple sheets

C) `.xlsx` is plain text and human-readable

D) Both are identical; only the extension differs

Q8. A1:A5 contains 10, `N/A` (text), 20, an empty cell, 30. What does `=AVERAGE(A1:A5)` return?

A) 15

B) 12

C) 20

D) `#DIV/0!`

Section B — Networking, Security & Cloud (12)

Q9. How many **usable host addresses** does a `/21` subnet provide?

A) 2048

B) 2044

C) 2046

D) 1022

Q10. What is the **network address** of `172.16.67.200/20`?

A) 172.16.0.0

B) 172.16.67.0

C) 172.16.80.0

D) 172.16.64.0

Q11. What is the **broadcast address** of `10.20.30.40/22`?

A) 10.20.31.255

B) 10.20.32.255

C) 10.20.30.255

D) 10.20.63.255

Q12. Which address is **NOT** in the same subnet as `192.168.10.35/27`?

A) 192.168.10.33

B) 192.168.10.62

C) 192.168.10.64

D) 192.168.10.40

Q13. Most specific summary route covering `172.16.0.0/24` … `172.16.7.0/24`?

A) 172.16.0.0/21

B) 172.16.0.0/22

C) 172.16.0.0/20

D) 172.16.0.0/16

Q14. Where does TLS operate in the OSI model?

A) Between Network and Transport

B) Between Transport and Application

C) Between Session and Presentation

D) Between Physical and Data Link

Q15. In TCP teardown, which host ends up in the `TIME_WAIT` state?

A) The host that receives the first FIN (passive close)

B) The host that sends the first FIN (active close), after sending the final ACK

C) Both hosts, always

D) Neither — only `CLOSE_WAIT` exists

Q16. With an `ECDHE` TLS cipher suite, how is the symmetric session key established?

A) The client generates it and encrypts it with the server's RSA public key

B) The server sends it inside the ServerHello

C) A pre-shared static key stored in the certificate

D) Both sides independently derive it from the ECDH exchange; it is never transmitted

Q17. A **digital signature** (signing a hash with the sender's private key) provides:

A) Confidentiality only

B) Integrity + authenticity + non-repudiation

C) Availability and confidentiality

D) Encryption of the message body

Q18. A logged-in bank user visits a malicious page that silently submits a fund-transfer request to the bank site. This is:

A) Stored XSS

B) CSRF

C) SQL injection

D) DNS cache poisoning

Q19. Your team uses a **managed cloud database service** (PaaS). Who patches the underlying OS and database engine?

A) The customer

B) The cloud provider

C) The ISP

D) The customer patches the OS, the provider patches the engine

Q20. Which port/protocol pairing is fully correct?

A) SNMP — UDP 161, Syslog — UDP 514

B) LDAPS — UDP 636, Syslog — TCP 514

C) LDAP — TCP 636, SNMP — UDP 162

D) SNMP — TCP 161, LDAPS — UDP 389

Section C — CS Core: DSA / DBMS / OOP / HTML-CSS-JS (12)

Q21. Time complexity of:

```
i ← n
WHILE i > 0
    FOR j ← 1 TO i  (O(1) work)
    i ← i DIV 2
END WHILE
```

A) O(n log n)

B) O(log n)

C) O(n)

D) $O(n^2)$

Q22. An LRU cache needs **O(1)** `get` and `put`. The standard structure is:

A) Balanced BST only

B) Hash map + doubly linked list

C) Array + binary search

D) Stack + queue

Q23. Which of these is **not stable** in its standard form?

A) Merge sort

B) Quick sort

C) Insertion sort

D) Bubble sort

Q24. You are given a pointer to a node **X** (not the tail) in a singly linked list, with no head pointer. How do you delete X in O(1)?

A) Traverse from the head to find the predecessor

B) Set X ← NULL

C) Copy X.next's data into X, then unlink and free X.next

D) It is impossible without the head

Q25. `R(A,B,C,D)` with FDs `A`→`B`, `B`→`C`, `A`→`D`; primary key `A`. Which normal form is violated?

A) 1NF

B) 2NF

C) 3NF

D) None — it is fully normalised

Q26. `Employees` has 10 rows; `dept` is NULL in 3 of them. What does `SELECT COUNT(*), COUNT(dept) FROM Employees` return?

A) 10 and 10

B) 7 and 7

C) 10 and 7

D) 10 and NULL

Q27. `SELECT ... FROM A LEFT JOIN B ON A.id = B.id WHERE B.status = 'X'` — the effect of the WHERE clause is:

A) None — all rows of A are still returned

B) It produces a syntax error

C) It converts it into a FULL OUTER join

D) It converts the join into an INNER join, discarding unmatched A rows

Q28. Which isolation level **permits dirty reads**?

A) Serializable

B) Read Committed

C) Repeatable Read

D) Read Uncommitted

Q29. Java overload resolution — with `f(int)`, `f(long)` and `f(Integer)` defined, which is chosen for `f(10)`?

A) `f(long)` — widening is always preferred

B) `f(int)` — exact match in phase 1, before boxing/widening

C) `f(Integer)` — boxing is tried first

D) Ambiguous — compile-time error

Q30. CSS — for `<div id="main" class="content"><p>text</p></div>` with:

```
#main p { color: red; }
.content p { color: blue; }
p { color: green; }
```

A) green

B) red

C) blue

D) Depends on source order only

Q31. Output of:

```
for (var i = 1; i <= 3; i++) { setTimeout(() => console.log(i), 0); }
```

A) 4 4 4

B) 1 2 3

C) 3 3 3

D) 1 1 1

Q32. The 8-bit two's-complement value `1111 0001` is:

A) 241

B) –113

C) –15

D) 15

Section D — Pseudocode & Code Tracing (14)

Q33.

```
count ← 0
FOR i ← 1 TO 4
    FOR j ← 1 TO i
        IF (i * j) MOD 3 = 0
            CONTINUE
        END IF
        count ← count + 1
    END FOR
END FOR
PRINT count
```

A) 10

B) 7

C) 6

D) 4

Q34.

```
FUNCTION G(n)
    IF n <= 1
        RETURN 1
    RETURN G(n-1) + G(n-2)
END FUNCTION
PRINT G(5)
```

A) 5

B) 13

C) 7

D) 8

Q35.

```
FUNCTION modify(a BY VALUE, b BY REFERENCE)
    a ← a + 10
    b ← b + 10
    RETURN a + b
END FUNCTION
x ← 5
y ← 5
r ← modify(x, y)
PRINT x, y, r
```

A) 15 15 30

B) 5 15 30

C) 5 5 30

D) 15 15 20

Q36. Standard bubble sort (ascending) on `A ← [5, 1, 4, 2, 8]`, counting every swap. How many swaps occur?

A) 5

B) 3

C) 4

D) 10

Q37.

```
A ← [2, 5, 8, 12, 16, 23, 38, 45, 67]
target ← 45
low ← 0; high ← 8; steps ← 0
WHILE low <= high
    steps ← steps + 1
    mid ← (low + high) DIV 2
    IF A[mid] = target
        BREAK
    ELSE IF A[mid] < target
        low ← mid + 1
    ELSE
        high ← mid - 1
    END IF
END WHILE
PRINT steps
```

A) 2

B) 4

C) 5

D) 3

Q38. (C/Java semantics, left-to-right) `x ← 5`, then `y ← x++ + ++x + x++`. What are `x` and `y`?

A) x = 8, y = 18

B) x = 7, y = 19

C) x = 8, y = 19

D) x = 6, y = 18

Q39.

```
count ← 0
FUNCTION H(n)
    count ← count + 1
    IF n <= 0
        RETURN 0
    RETURN n + H(n-2)
END FUNCTION
PRINT H(6), count
```

A) 12 3

B) 0 4

C) 18 4

D) 12 4

Q40.

```
FUNCTION swap(a, b)          // both by value
    temp ← a
    a ← b
    b ← temp
END FUNCTION
p ← 3
q ← 7
swap(p, q)
PRINT p, q
```

A) 7 3

B) 3 3

C) 3 7

D) Runtime error

Q41.

```
s ← "aB3cD9eF1"
out ← ""
FOR each ch IN s
    IF ch is a digit
        out ← ch + out
    END IF
END FOR
PRINT out
```

A) 391

B) 193

C) aBcDeF

D) 931

Q42.

```
FUNCTION check(n)
    PRINT "check"
    RETURN n > 0
END FUNCTION
a ← 0
IF (a <> 0) AND (10 DIV a > 2)
    PRINT "YES"
ELSE
    PRINT "NO"
END IF
```

A) YES

B) Runtime error (division by zero)

C) NO

D) check NO

Q43.

```
a ← 48; b ← 18
WHILE b <> 0
    t ← b
    b ← a MOD b
    a ← t
END WHILE
PRINT a
```

A) 12

B) 18

C) 0

D) 6

Q44.

```
M ← [[1,2,3],[4,5,6],[7,8,9]]
sum ← 0
FOR i ← 0 TO 2
    sum ← sum + M[i][2 - i]
END FOR
PRINT sum
```

A) 12

B) 15

C) 18

D) 25

Q45.

```
n ← 7; steps ← 0
WHILE n <> 1
    steps ← steps + 1
    IF n MOD 2 = 0
        n ← n DIV 2
    ELSE
        n ← 3 * n + 1
    END IF
END WHILE
PRINT steps
```

A) 17

B) 15

C) 16

D) 11

Q46.

```
A ← [4, 9, 2, 9, 7]
max1 ← -1; max2 ← -1
FOR each v IN A
    IF v > max1
        max2 ← max1
        max1 ← v
    ELSE IF v > max2 AND v <> max1
        max2 ← v
    END IF
END FOR
PRINT max1, max2
```

A) 9 9

B) 9 4

C) 7 9

D) 9 7

Section E — Applied: SQL & Code Output (4)

Use this table for Q47–Q48:

`Employees(emp_id, name, dept, salary)`

emp_id	name	dept	salary
1	Amit	HR	50000
2	Bala	IT	70000
3	Chitra	IT	60000
4	Dev	HR	45000
5	Esha	IT	70000

Q47.

```
SELECT dept, COUNT(*) AS n, AVG(salary) AS avg_sal
FROM Employees
GROUP BY dept
HAVING COUNT(*) > 1
ORDER BY avg_sal DESC;
```

A) 2 rows; first row is HR

B) 5 rows

C) 2 rows; first row is IT

D) Error — `COUNT(*)` cannot be used in `HAVING`

Q48. Which query lists departments having **more than 2 employees**?

A) `SELECT dept FROM Employees WHERE COUNT(*) > 2 GROUP BY dept;`

B) `SELECT dept FROM Employees GROUP BY dept WHERE COUNT(*) > 2;`

C) `SELECT dept FROM Employees GROUP BY dept HAVING COUNT(*) > 2;`

D) `SELECT dept, COUNT(*) FROM Employees HAVING COUNT(*) > 2;`

Q49. Python:

```
a = [1, 2, 3]
b = a
b.append(4)
c = a[:]
c[0] = 9
print(len(a), a[0], len(c))
```

A) 3 1 3

B) 4 9 4

C) 4 9 3

D) 4 1 4

Q50. JavaScript (three elements have class `item`):

```
const cols = document.getElementsByClassName('item');
cols.forEach(el => el.classList.add('active'));
```

A) All three elements get the class `active`

B) Silently does nothing

C) `TypeError: cols.forEach is not a function` — it is a live HTMLCollection, not an array

D) Only the first element gets the class

ANSWER KEY

Do not look here until you have finished all 50 questions.

— Answer Key for Set 1 Do not scroll here until you have finished all 50 questions and written your answers down.

Quick grid

Q	1	2	3	4	5	6	7	8	9	10
Ans	C	A	D	C	D	B	B	C	C	D

Q	11	12	13	14	15	16	17	18	19	20
Ans	A	C	A	B	B	D	B	B	B	A

Q	21	22	23	24	25	26	27	28	29	30
Ans	C	B	B	C	C	C	D	D	B	B

Q	31	32	33	34	35	36	37	38	39	40
Ans	A	C	C	D	B	C	D	C	D	C

Q	41	42	43	44	45	46	47	48	49	50
Ans	B	C	D	B	C	D	C	C	D	C

Explanations

Q1 — C. With `TRUE` (or omitted), VLOOKUP does an *approximate* match using binary search and requires the first column sorted **ascending**. Unsorted data silently returns a wrong-but-plausible answer. Use `FALSE`/`0` for exact match.

Q2 — A. `$A4` = column locked, row relative → C4→D5 (1 right, 1 down) gives `$A5`. `B$2` = column relative, row locked → `C$2`. Result `=$A5+C$2`.

Q3 — D. Values in [8,15] → 12, 8, 15 → **3**. COUNTIFS ANDs the criteria pair-wise.

Q4 — C. `FALSE` with no exact match → `#N/A`, but `IFERROR` catches it and returns `"Not Found"`.

Q5 — D. *Hide Background Graphics* (Format Background) suppresses everything inherited from the master (logo, shapes) on that one slide while keeping the design/background.

Q6 — B. *Compare* shows the differences between two documents as a third, tracked-change document; ***Combine*** merges multiple reviewers' edits into one document. You want both sets of edits merged → Combine.

Q7 — B. CSV = plain text, one table, comma-separated, no formulas/formatting/multiple sheets/macros. `.xlsx` is a zipped OOXML workbook.

Q8 — C. AVERAGE **ignores** text and blanks *inside a range* → (10+20+30)/3 = **20**. (`COUNTA` would count the text; text passed *directly* as an argument gives `#VALUE!`.)

Q9 — C. /21 → 11 host bits → $2^{11} - 2$ = **2046**.

Q10 — D. /20 = 255.255.240.0 → third-octet block size 16 → 67 falls in 64–79 → **172.16.64.0**.

Q11 — A. /22 = 255.255.252.0 → third-octet block 4 → 30 falls in 28–31 → broadcast **10.20.31.255**.

Q12 — C. /27 block size 32 → 35 is in 32–63. `.33`, `.40`, `.62` are inside; `.64` is the network address of the next subnet.

Q13 — A. Eight consecutive /24s → 3 bits → 24 – 3 = **/21**. `/16` also covers them but is not the *most specific*.

Q14 — B. TLS sits above Transport (TCP) and below the Application protocols it protects — a session/presentation-layer service operating between Transport and Application.

Q15 — B. The **active closer** (first FIN sender) sends the last ACK and waits in `TIME_WAIT` (2 × MSL) to absorb delayed duplicate packets. The passive side passes through `CLOSE_WAIT` → `LAST_ACK`.

Q16 — D. With (EC)DHE the master secret is **derived by both peers**; the server only *signs* its ephemeral public parameter with the certificate's private key for authentication. Encrypting a client-generated key with RSA is the older, non-forward-secret RSA key transport.

Q17 — B. Signing a hash with a private key proves who signed it and that it wasn't altered (integrity + authenticity + non-repudiation). It does **not** give confidentiality — the message stays readable.

Q18 — B. CSRF — the attacker makes the victim's *browser* send an authenticated request to a site where the victim is already logged in. XSS injects script into a trusted page; the key here is the cross-site **forged request**.

Q19 — B. In PaaS (managed DB) the provider owns OS, patching, engine upgrades and backups; the customer owns data, schema, access control and retention settings. In IaaS the customer patches the guest OS.

Q20 — A. SNMP 161/UDP, Syslog 514/UDP. LDAP = 389, LDAPS = **636** (TCP), SNMP traps = **162**.

Q21 — C. Inner loop runs n + n/2 + n/4 + … = 2n – 1 → **O(n)**. The halving outer loop alone is O(log n), but the work per pass shrinks geometrically.

Q22 — B. Hash map gives O(1) lookup; doubly linked list gives O(1) move-to-front/evict. A BST alone gives O(log n), not O(1).

Q23 — B. Quick sort's partition swaps destroy the relative order of equal keys. Merge, insertion, bubble and counting sort are stable (heap and selection are not).

Q24 — C. Classic O(1) trick: copy the successor's payload into X, then bypass the successor. (Fails if X is the tail — then you need the head.)

Q25 — C. `A→B→C` is a **transitive dependency** of a non-key attribute on another non-key attribute → violates **3NF**. With a single-attribute key there is no partial dependency, so 2NF holds.

Q26 — C. `COUNT(*)` counts rows = 10. `COUNT(col)` counts **non-NULL** values in that column = 7.

Q27 — D. Unmatched left rows have `B.status = NULL`; `NULL = 'X'` is UNKNOWN → those rows are dropped, so the LEFT JOIN silently degrades to an INNER join. To keep them, put the filter in the `ON` clause.

Q28 — D. Read Uncommitted allows dirty reads. Read Committed blocks them (allows non-repeatable reads); Repeatable Read blocks those (allows phantoms); Serializable blocks all.

Q29 — B. Java resolves overloads in three phases: **(1) no boxing/widening — exact match `f(int)` wins**, (2) widening + boxing, (3) varargs.

Q30 — B. Specificity: `#main p` = (1 id, 0 class, 1 element) beats `.content p` = (0,1,1) beats `p` = (0,0,1). Ids dominate regardless of source order → **red**.

Q31 — A. `var` is function-scoped: one shared `i`. By the time the callbacks run, the loop has finished and `i === 4`. Use `let` (block-scoped, fresh binding per iteration) to get 1 2 3.

Q32 — C. MSB = 1 → negative. Magnitude: invert `1111 0001` → `0000 1110` = 14, +1 = 15 → –**15**.

Q33 — C. Skipped pairs (product divisible by 3): (3,1),(3,2),(3,3),(4,3) → 4 skipped of 10 → **6** counted.

Q34 — D. The base case returns **1** (not n), so the series is 1,1,2,3,5,8… → `G(5) = 8`.

Q35 — B. `a` is by value → caller's `x` stays 5. `b` is by reference → caller's `y` becomes 15. Return = 15 + 15 = 30 → `5 15 30`.

Q36 — C. Bubble-sort swaps = number of **inversions**: (5,1),(5,4),(5,2),(4,2) = **4**.

Q37 — D. mid 4 (=16) → low 5; mid 6 (=38) → low 7; mid 7 (=45) → found. **3** iterations.

Q38 — C. Left to right: `x++` yields 5 (x→6); `++x` → x = 7, yields 7; `x++` yields 7 (x→8). `y = 5+7+7 = 19`, `x = 8`.

Q39 — D. Calls: H(6), H(4), H(2), H(0) → **4** calls; sum = 6+4+2+0 = **12**.

Q40 — C. Pass-by-value: the function swaps its local copies. `p` and `q` are unchanged → `3 7`.

Q41 — B. Digits met in order 3, 9, 1 and each is **prepended**: "3" → "93" → "193".

Q42 — C. `AND` short-circuits: `a <> 0` is false, so `check()` is never called (no "check" printed) and `10 DIV a` is never evaluated → no division by zero → prints **NO**.

Q43 — D. Euclid: (48,18) → (18,12) → (12,6) → (6,0) → **6**.

Q44 — B. Anti-diagonal: M[0][2]=3 + M[1][1]=5 + M[2][0]=7 = **15**. (The main diagonal is also 15 here — check which index you traced!)

Q45 — C. 7→22→11→34→17→52→26→13→40→20→10→5→16→8→4→2→1 = **16** steps.

Q46 — D. First 9 sets max1 = 9, max2 = 4. The **second** 9 fails both tests (`v > max1` false, `v <> max1` false), so it never becomes max2. Then 7 > 4 and 7 ≠ 9 → max2 = 7 → `9 7`.

Q47 — C. HR (n = 2, avg 47500) and IT (n = 3, avg 66666.67) both pass `HAVING COUNT(*) > 1` → 2 rows; `DESC` puts **IT** first.

Q48 — C. `WHERE` filters rows *before* grouping and cannot contain aggregates; `HAVING` filters groups. (D) counts over the whole table, not per department.

Q49 — D. `b = a` aliases the same list → the append affects `a`. `c = a[:]` is a **shallow copy** → changing `c[0]` leaves `a[0] = 1` → `4 1 4`.

Q50 — C. `getElementsByClassName` returns a **live HTMLCollection**, which has no `forEach`. Use `querySelectorAll('.item')` (static NodeList, has `forEach`) or `Array.from(cols).forEach(...)`.

Score guide (Set 1)

Score	Verdict
43–50	Excellent — comfortably above the Accenture bar
34–42	Good — revise the topics you dropped

25–33	At risk — re-read the deep-dives (ch. 07) and re-sit this set in 3 days
< 25	Rebuild fundamentals (ch. 01–06) before attempting Set 2